

## 1 Solution Strategy

For this project, I implemented a Conflict-Driven Clause Learning (CDCL) SAT solver in Python, later optimized with Cython. I then added some specific augmentations to generic CDCL drawing inspiration from the MiniSAT solver [Sorensson and Een \(2005\)](#); [Sörensson and Claessen \(2026\)](#). After reading a survey paper on restart strategies [Haim and Heule \(2014\)](#), I spent (perhaps too much) time thinking about different restart ideas. Most of these did not pan out. Eventually, I ended up with an implementation with the following core components:

- I used Variable State Independent Decaying Sum (VSIDS) for choosing branching variables. I learned about this starting at a set of lecture slides on SAT Heuristics [Clarke](#), from which I learned about the Chaff solver [Moskewicz et al. \(2001\)](#). My VSIDS implementation is not as sophisticated as the Chaff solver, but has the general flavor.
- Conflict analysis: I used a First Unique Implication Point (1-UIP) scheme, that I learned about from a set of lecture slides from the CIS189 course at the University of Pennsylvania [Menezes \(2024\)](#).
- Restart strategies: Luby sequence-based restarts to escape poor search regions [Luby et al. \(1993\)](#); [Haim and Heule \(2014\)](#).

## 2 Learning

### 2.1 What Worked

My biggest wins in terms of speedup were:

**VSIDS Heuristic** provided by far the biggest speedup, especially when compared to my initial naive "next unassigned variable" decision strategy.

**The Luby restart sequence** which provides short initial probes that grow geometrically. This helped balance exploration of the search space and exploitation of the value of lots of fast initial restarts [Haim and Heule \(2014\)](#).

**Negative Branching Direction** Following MiniSat's approach, I always assign the decision variables to false. Haim et al [Haim and Heule \(2014\)](#) show this is better than arbitrary, and that false branching empirically outperforms true branching. This simple heuristic provided noticeable speedups versus random or true heuristics.

**Low-Level Representational Changes** After compiling the Python source to C via Cython, I implemented several representational changes to optimize the C-level execution. This primarily involved replacing "boxed" Python structures (e.g., `dict`, `Optional`) with raw C-style constructs. Specifically, the variable assignment mapping was transitioned from a dictionary-based structure,

assignments: Dict[int, Optional[bool]], to a contiguous integer array, assignments: List[int]. In this representation, an assignment for a variable  $v$  is defined as 0 if unassigned, 1 if True, and -1 if False.

## 2.2 What Did Not Work

I spent *most* of my time trying to tune instance-adaptive hyperparameters based on clause-to-variable density. This approach was inspired by portfolio solvers like SATzilla [Xu et al. \(2008\)](#), which select different solvers based on instance features. My adaptation applies the same principle within a single solver by switching hyperparameters at startup as well as learning about how clause density could affect the probability of a formula’s SAT or UNSAT-ness [Dudek et al. \(2017\)](#).

Initially, I attempted to use a phase transition heuristic based on the known 3-SAT phase transition at density 4.26 [Dudek et al. \(2017\)](#). I hypothesized that this could guide restart strategies or other parameters. However, this didn’t help in practice—the harder instances in the test suite weren’t hitting this specific density point. I eventually ended up classifying instances into three broad clusters based on their clause density, and assigned roughly the following hyperparams:

- Density  $\leq 6$ : Likely SAT, Slow variable decay (0.99), large Luby base (512), and low randomization (0.005).
- Density (6–30): Probably in the phase-transition region and hard to solve (decay 0.95, Luby 100, random 0.02)
- High density ( $> 30$ ): Likely UNSAT. Aggressive decay (0.85), frequent restarts (Luby 32), and high randomization (0.08)

Unfortunately, most of the harder CNF files I tested tended to be in the medium density / phase transition range. High and low density instances are already effectively dealt with by clause learning. There’s no real reason to optimize for them. At the same time, I do feel like there were modest gains from this approach.

**Giving UNSAT for Extreme Density** I briefly considered returning UNSAT immediately for extremely high-density instances, trading completeness for speed. While this would be correct with high probability and extremely fast on some UNSAT cases, I ultimately rejected this approach because it would make the solver incomplete—an unacceptable compromise for a correct implementation.

## References

- Edmund M. Clarke. Heuristics for efficient SAT solving. Lecture slides, Carnegie Mellon University. URL [https://www.cs.cmu.edu/~emc/15-820A/reading/sat\\_cmu.pdf](https://www.cs.cmu.edu/~emc/15-820A/reading/sat_cmu.pdf). Accessed: 2026-02-15.
- Jeffrey M. Dudek, Kuldeep S. Meel, and Moshe Y. Vardi. Combining the  $k$ -cnf and xor phase-transitions, 2017. URL <https://arxiv.org/abs/1702.08392>.

- Shai Haim and Marijn Heule. Towards ultra rapid restarts. *arXiv preprint arXiv:1402.4413*, 2014.
- Michael Luby, Alistair Sinclair, and David Zuckerman. Optimal speedup of las vegas algorithms. *Information Processing Letters*, 47(4):173–180, 1993.
- Rohan Menezes. Lecture 4: Modern techniques in SAT solving. University of Pennsylvania, CIS 1890 Course Materials, 2024. URL <https://www.cis.upenn.edu/~cis1890/files/Lecture4.pdf>. Accessed: February 15, 2026.
- Matthew W Moskewicz, Conor F Madigan, Ying Zhao, Lintao Zhang, and Sharad Malik. Chaff: Engineering an efficient sat solver. In *Proceedings of the 38th annual Design Automation Conference*, pages 530–535, 2001.
- Niklas Sörensson and Koen Claessen. MiniSat: A minimalistic and high-performance SAT solver. <https://github.com/niklasso/minisat>, 2026. Accessed: 2026-02-15.
- Niklas Sorensson and Niklas Een. Minisat v1. 13-a sat solver with conflict-clause minimization. *SAT*, 53(2005):1–2, 2005.
- Lin Xu, Frank Hutter, Holger H Hoos, and Kevin Leyton-Brown. Satzilla: portfolio-based algorithm selection for sat. *Journal of artificial intelligence research*, 32:565–606, 2008.